Republic of the Philippines

North Eastern Mindanao State University

ByteQuest: A Gamified Programming Learning Platform Using Comparative Machine Learning Models for Predicting Programming-Task Performance

An Undergraduate Thesis
Presented to the faculty of College of Information Technology Education
NORTH EASTERN MINDANAO STATE UNIVERSITY
Tandag City, Surigao Del Sur

In Partial Fulfillment
of the Requirements for the Degree of
Bachelor of Science in Computer Science

By:

Dagohoy, Venz Andrew P.
Loren, Leandro Kieser Y.
Mangin, Eliamen A.
Somogba, Mares Susette V.

October 2026

Chapter I

INTRODUCTION

Background of the Study

Computer programming is an important component of computer science education because students are expected to understand programming concepts, develop solutions, and translate those solutions into executable code. However, learning programming can be challenging because students must develop problem-solving, algorithmic, and coding skills at the same time. Amnouychokanant et al. (2021) noted that learning programming in higher education can be difficult for beginners because they must express solutions in a form that a computer can execute, while syntax and the selection of appropriate control structures can also present difficulties. Preliminary interviews conducted by the researchers with selected Computer Science students likewise indicated difficulties involving programming concepts, debugging, algorithms, and maintaining consistent practice. For example, one student described Java as difficult because of syntax-related issues, while another identified loops as an area of difficulty. Students also described different approaches to practicing and debugging their programs, including independently examining errors and using

artificial intelligence tools for assistance. Other interview responses indicated that access to a computer can affect opportunities for practice and that students may continue working when motivated to complete a task but stop when they become mentally tired. These preliminary observations do not establish that all Computer Science students experience the same difficulties, but they provide local context for examining a learning platform that offers structured programming practice and collects performance data. The study is situated within the Bachelor of Science in Computer Science program of North Eastern Mindanao State University (NEMSU).

One approach examined in programming education is gamification, which incorporates game-related elements into learning activities. Gamification may include points, badges, achievements, leaderboards, challenges, progress indicators, and feedback. Zhan et al. (2022), in a meta-analysis of 21 empirical studies, reported positive overall effects of gamification in programming education, particularly for motivation and academic achievement, although the effects varied across implementations and learning contexts. Philippine studies also show the use of gamification in programming education. Garcia and Revano (2021) evaluated a gamified Python programming course in higher education and reported higher laboratory skills performance for the experimental group, together with improvements in attitude and self-efficacy. Gonzaga and Badiang (2025) developed and evaluated Quackademy, a gamified mobile learning tool for public senior high school students in Region XI that addressed selected programming

competencies through elements such as points, badges, and an interactive game. These studies demonstrate that gamification has been applied to programming education in both international and Philippine settings, but the reviewed local studies do not directly address the integration of gamified programming activities with prediction of a student's outcome on a subsequent programming task.

Another relevant direction is the use of automated assessment, machine learning, and AI-assisted content generation in programming education. Automated programming assessment can evaluate submitted code using predefined test cases, allowing programming exercises to be assessed systematically and supporting feedback during practice. Barczak et al. (2023), for example, described an automated assessment system integrated into a data structures and algorithms course. Machine learning can be applied to student performance and interaction data to identify patterns associated with programming outcomes. Zhang et al. (2025) demonstrated a machine-learning approach for predicting student progress in online programming education, while Durak and Bulut (2024) compared several classification algorithms, including Logistic Regression and Random Forest, for predicting programming performance. In the proposed platform, student-submitted programs will be executed in a controlled environment and evaluated against predetermined test cases to determine the actual outcome of a programming task, whereas Logistic Regression and Random Forest will use previously collected performance-related features to predict

whether a student will successfully complete a subsequent programming task. The platform will also include a document-based learning-content generation feature in which students or instructors can upload programming-related PDF materials. Research on educational question generation indicates that neural and language-model approaches can generate questions from learning materials, while also emphasizing content relevance, answer alignment, difficulty control, and human evaluation (Al Faraby et al., 2024). For document-grounded generation, the proposed platform will use a Retrieval-Augmented Generation (RAG) process in which relevant portions of the uploaded material are retrieved and provided as contextual information to the language model. A recent systematic survey identified educational content development and assessment as important applications of RAG while also noting challenges such as hallucination and the need for reliable retrieved knowledge (Li et al., 2025). The proposed platform will use Gemini 2.5 Flash as the large language model for this content-generation component. Borowitz et al. (2026) demonstrated a comparable document-grounded workflow in which Gemini 2.5 Flash was used with a PDF source to generate educational multiple-choice questions, with generated questions undergoing review and revision before evaluation.

Despite existing research on gamified programming environments, automated programming assessment, student modeling, machine-learning-based prediction, and AI-assisted educational content generation, there remains a context-specific

opportunity to integrate these components within one programming learning platform for selected Computer Science students at NEMSU. Existing Philippine studies have examined gamified programming learning, while international studies have demonstrated automated assessment, predictive modeling, and document-grounded content generation. However, these studies differ in their populations, programming contexts, system designs, variables, and research objectives. The proposed study therefore addresses a more specific gap by developing ByteQuest, a gamified programming learning platform that supports Python, C++, and Java while aligning programming tasks with the programming languages and concepts that participating students have already encountered through their coursework; provides document-based generation of quizzes, review materials, and programming challenges from uploaded instructional PDFs using RAG and Gemini 2.5 Flash; evaluates programming submissions through automated test cases; collects student performance data; and compares Logistic Regression and Random Forest for predicting the outcome of a subsequent programming task among selected NEMSU Computer Science students. The study does not assume that either predictive model will perform better before evaluation. Instead, their performance will be empirically compared using appropriate classification measures such as accuracy, precision, recall, and F1-score. Generated learning materials will not be treated as automatically correct and will require pedagogical content review and, where applicable, instructor review or editing because recent research identifies the need to examine the relevance, alignment, and quality of

AI-generated educational questions.

Theoretical Framework

The study is primarily anchored on Gamification Theory, Learning Analytics/Educational Data Mining, and the concept of Automated Programming Assessment.

Gamification Theory

Gamification refers to the use of game design elements in non-game contexts. In educational environments, these elements may include points, badges, leaderboards, challenges, progress indicators, and feedback. The purpose of incorporating these elements into ByteQuest is not to assume that gamification automatically improves programming ability, but to provide a structured environment in which students can participate in programming activities and generate observable learning-performance data. Zhan et al. (2022) found positive effects of gamification in programming education, particularly for motivation and academic achievement, although effects varied according to implementation and learning context.

In ByteQuest, gamification will therefore serve as the learning environment rather than the predictive model. Students can complete quizzes, programming challenges, quests, and other activities while receiving points, progress indicators,

badges, or similar game elements.

Learning Analytics and Machine Learning

Learning analytics involves using learner-generated data to understand and support learning processes. In programming education, student interactions and performance data can be used to identify patterns associated with progress or difficulty. Zhang et al. (2025), for example, demonstrated the use of machine learning to predict student progress in online programming education using course interaction and programming-task features. Durak and Bulut (2024) likewise demonstrated the use of classification algorithms, including Logistic Regression and Random Forest, to predict programming performance.

ByteQuest applies this principle by collecting structured performance information from programming activities. These data will be transformed into features that can be used to train and evaluate two classification models:

1. Logistic Regression
2. Random Forest

The models will predict a predefined binary outcome for a subsequent programming task, such as successful completion or unsuccessful completion based on the study's operational definition.

The purpose is not to label a student's overall programming ability. Rather, the models will estimate the likelihood of the defined outcome for a subsequent

programming task based on previously observed data.

Automated Programming Assessment

Automated programming assessment allows submitted code to be evaluated using predefined test cases. This provides an objective way to determine whether a submitted solution meets specified programming requirements. Barczak et al. (2023) demonstrated an automated assessment system in programming coursework and reported its use for formative learning and feedback.

In ByteQuest, automated assessment will determine programming-task outcomes. These outcomes will then become part of the performance data used by the predictive models.

AI-Assisted Learning-Content Generation

The study also incorporates an AI-assisted learning-content generation component. When a programming-related PDF is uploaded, the system will extract and organize its content, retrieve relevant portions through a Retrieval-Augmented Generation (RAG) process, and provide the retrieved content to Gemini 2.5 Flash as the large language model for generation. The system can then generate quizzes, review questions, and programming challenges based on the uploaded material. Li et al. (2025) identified educational content development and assessment as applications of RAG and emphasized the importance of retrieval quality and mitigation of hallucinated content. Borowitz et al. (2026) also

demonstrated the use of Gemini 2.5 Flash with a PDF source for generating educational multiple-choice questions, with a review process used to revise generated items.

Research on automatic educational question generation further supports the use of neural and language-model approaches for producing questions from learning materials, while emphasizing content selection, relevance, answer alignment, difficulty control, and human evaluation (Al Faraby et al., 2024). Therefore, ByteQuest will treat generated content as AI-assisted material that requires pedagogical content review rather than assuming that every generated question is automatically correct.

Conceptual Framework

The study will use the Input–Process–Output (IPO) model as its conceptual framework.

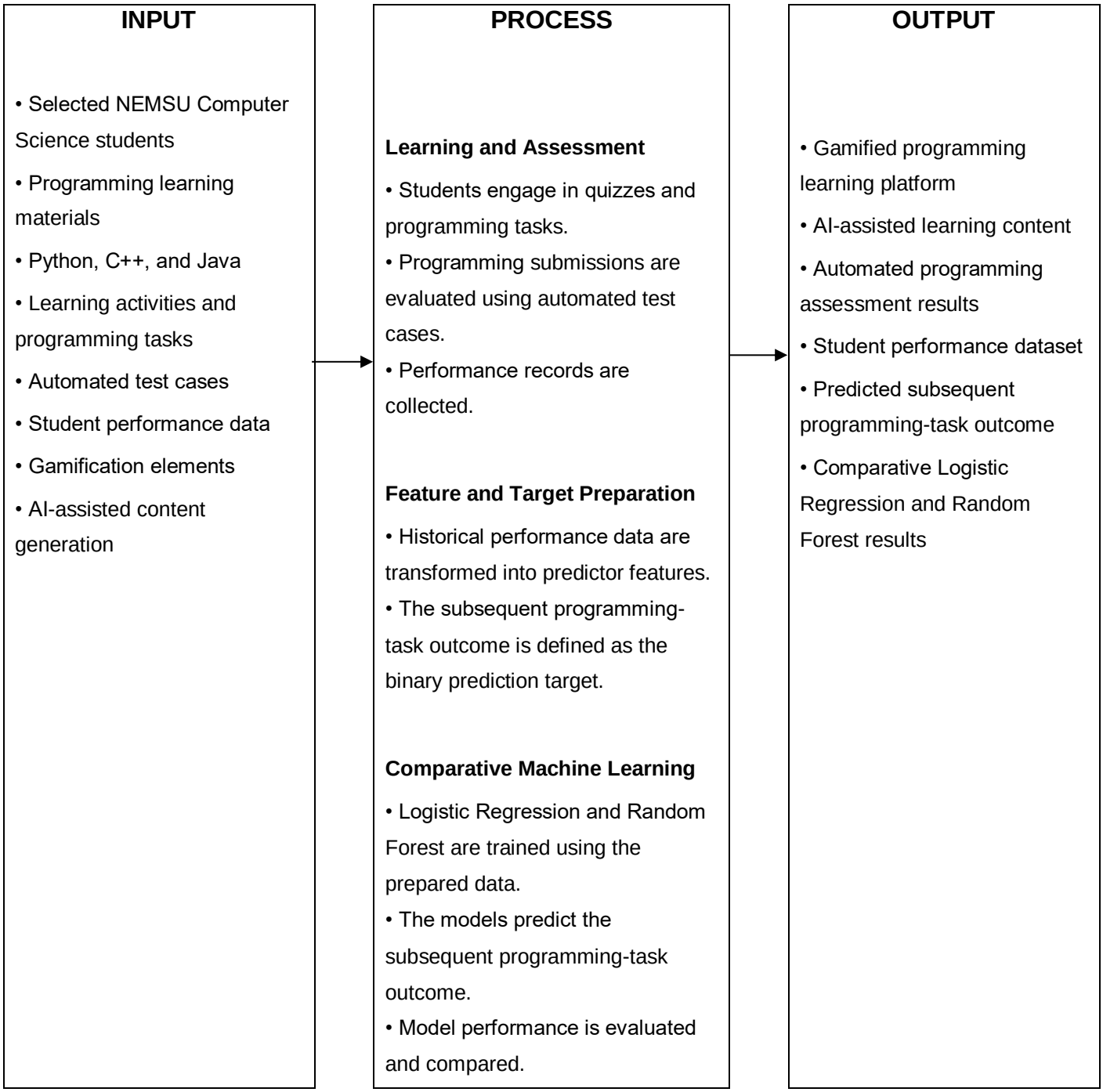


Figure 1: *Input-Process-Output (IPO) Model of the ByteQuest System*

Input

The **Input** consists of the participants, learning resources, programming activities, assessment components, and student performance information required by the study. The selected NEMSU Computer Science students serve as the intended participants. The learning resources include programming materials covering **Python, C++, and Java**, while the learning activities include quizzes and programming tasks. Test cases are included to support the assessment of submitted programming solutions. Student performance data obtained from completed learning activities and programming tasks provide the historical information needed for the machine-learning component. Gamification elements and AI-assisted content generation are also included as supporting components of the proposed learning platform.

Process

The **Process** describes how the identified inputs are transformed into the expected outputs. First, students engage with the learning activities and programming tasks provided through the proposed platform. Programming submissions are assessed using automated test cases, and the resulting performance information is recorded. The collected historical performance data are then prepared as features for the machine-learning models, while the outcome of a subsequent programming task serves as the prediction target. The prepared data

are used to develop and evaluate two comparative machine-learning models, **Logistic Regression and Random Forest**, for predicting whether a student will successfully complete a subsequent programming task. The models are then evaluated using the selected performance measures to determine their predictive performance. This follows the study's distinction between automated assessment, which determines the actual programming-task outcome, and machine-learning prediction, which uses information available before the subsequent task.

Output

The **Output** represents the results produced from the processing of the identified inputs. These include the proposed gamified programming learning platform, AI-assisted learning content, automated programming assessment results, and the student performance dataset collected through the learning activities. The machine-learning component produces predictions of the outcome of subsequent programming tasks and provides comparative results for **Logistic Regression and Random Forest** based on the selected evaluation measures. The output therefore represents the combined results of the learning, assessment, and machine-learning processes of the study.

Connection of the IPO Components

The three components of the framework are connected sequentially. The **Input** provides the participants, learning resources, programming activities,

assessment components, and performance information needed by the study. These inputs are processed through the **Process**, where students participate in learning activities, programming submissions are assessed, historical performance data are prepared, and the two machine-learning models are applied and evaluated. The results of these processes form the **Output**, which includes the proposed learning platform, learning and assessment results, and the comparative machine-learning results. Thus, the framework illustrates how the resources and information provided as inputs are transformed through learning, assessment, and predictive modeling to achieve the intended outputs of the study.

Statement of the Problem

The study aims to develop ByteQuest: A Gamified Programming Learning Platform Using Comparative Machine Learning Models for Predicting Programming-Task Performance of Selected NEMSU Computer Science Students.

Specifically, it seeks to answer the following questions:

1. What programming performance data and features can be collected from selected NEMSU Computer Science students while completing programming learning activities and tasks in Python, C++, and Java, with the assigned activities aligned with the programming languages and concepts they have already encountered through their coursework?

2. How can ByteQuest generate quizzes, review questions, and programming challenges from uploaded programming-related PDF materials using an LLM-based content-generation component?
3. How can automated test cases be used to determine the outcome of students' programming-task submissions in ByteQuest?
4. How accurately can Logistic Regression predict the outcome of a subsequent programming task based on previously collected student-performance features?
5. How accurately can Random Forest predict the outcome of a subsequent programming task based on the same performance features?
6. How do Logistic Regression and Random Forest compare in terms of accuracy, precision, recall, and F1-score when predicting the defined subsequent programming-task outcome?
7. How can the resulting predictive model be incorporated into ByteQuest to support the selection or sequencing of subsequent programming challenges?
8. How do the intended users evaluate ByteQuest in terms of functionality and usability?

Objectives of the Study

General Objective

The general objective of the study is to develop ByteQuest, a gamified programming learning platform that uses comparative machine-learning models to predict outcomes on subsequent programming tasks among selected NEMSU Computer Science students.

Specific Objectives

Specifically, the study aims to:

1. Collect and organize programming-performance data from selected NEMSU Computer Science students completing programming activities in Python, C++, and Java, with activities aligned with the programming languages and concepts they have already encountered through their coursework.
2. Develop an AI-assisted learning-content generation feature that extracts programming-related content from uploaded PDF materials and generates quizzes, review questions, and programming challenges.
3. Implement an automated programming assessment mechanism using predefined test cases to determine the outcome of programming-task submissions.
4. Identify and prepare performance-related features that can be used for subsequent-task prediction.

5. Develop and evaluate a Logistic Regression model for predicting the defined outcome of a subsequent programming task.
6. Develop and evaluate a Random Forest model using the same performance-related features.
7. Compare the performance of Logistic Regression and Random Forest using accuracy, precision, recall, and F1-score.
8. Integrate the evaluated predictive approach into ByteQuest to support the selection or sequencing of subsequent programming challenges.
9. Evaluate the developed ByteQuest platform in terms of its functionality and usability.

Scope and Delimitation of the Study

This study focuses on developing and evaluating ByteQuest, a gamified programming learning platform for selected Computer Science students at North Eastern Mindanao State University. The study will cover three programming languages: Python, C++, and Java. Programming activities will be assigned according to the programming languages and concepts that participating students have already encountered through their coursework. Based on the current programming-course sequence used in the study, the relevant progression includes C++ in the first semester of the first year, Python in the second semester

of the first year, and Java in the first semester of the second year.

The platform will provide programming learning activities, including quizzes, review materials, and programming challenges. It will also incorporate gamification elements such as points, progress indicators, badges, quests, or related mechanisms. The system will include a document-based content-generation feature that allows a student or instructor to upload a programming-related PDF. The system will process the uploaded document, retrieve relevant content through a RAG process, and provide the retrieved material to Gemini 2.5 Flash for generating quizzes, review questions, and programming challenges. Generated materials will not automatically be treated as error-free; they will require pedagogical content review and, where applicable, instructor review or editing.

The programming activities will cover concepts relevant to Python, C++, and Java. Programming submissions will be executed in a controlled environment and evaluated using predefined automated test cases. The resulting task outcome will be recorded as the target outcome and, where applicable, as a historical performance feature for subsequent-task prediction.

The machine-learning component will focus specifically on comparing Logistic Regression and Random Forest for predicting the outcome of a subsequent programming task. The study will use performance-related features

derived from activities completed before the target task. The target variable will be operationally defined as a binary outcome according to the study's assessment criteria. The study will not claim to predict a student's overall programming ability, future academic performance, graduation outcome, or general intelligence.

The machine-learning models will be evaluated using classification measures such as accuracy, precision, recall, and F1-score. The study will not assume beforehand that Logistic Regression or Random Forest will produce better results. Model performance will be determined from the collected dataset and evaluation procedure.

The study will use locally collected data from the selected participants rather than assuming that an existing public programming dataset represents NEMSU students. The final participant count and dataset size will be determined according to the approved research design, participant availability, and ethical requirements. Because multiple observations may come from the same student, the data-splitting procedure will be designed to reduce information leakage between training and evaluation data.

The study is limited to selected NEMSU Computer Science students and therefore will not automatically generalize its findings to all NEMSU students, all Computer Science students in the Philippines, or programming students in other

institutions. Generalizability will depend on the sampling procedure, participant characteristics, and the amount of data collected.

The study will also not compare multiple LLMs as a separate research objective. Gemini 2.5 Flash is included primarily as an AI-assisted learning-content generation component, while the principal comparative machine-learning investigation remains the comparison between Logistic Regression and Random Forest. Generated questions and challenges will require pedagogical content review because LLM-generated educational content may contain inaccuracies, weak alignment with the source material, or information that is not adequately supported by the retrieved content.

Significance of the Study

Students

The study may provide students with a structured environment for practicing programming through quizzes, review activities, programming challenges, and gamified learning elements, particularly for the BSCS (Bachelor of Science in Computer Science) students. The PDF-based content-generation feature may allow learning activities to be created from programming materials relevant to the student's current topic. The predictive component may also support sequencing or selecting subsequent programming challenges based on previously observed

performance.

Computer Programming Instructors

The platform may help instructors prepare programming learning activities and monitor student performance. The PDF-based generation feature may provide an initial set of quizzes and programming challenges that instructors can review and modify before publication. The performance data and predictive results may also provide additional information that instructors can consider when designing or sequencing programming activities.

North Eastern Mindanao State University

The study may provide a prototype learning platform specifically designed around the programming-learning context of selected NEMSU Computer Science students. It may also show how automated programming assessment, gamification, AI-assisted content generation, and learning analytics can be integrated into a single educational platform.

Future Developers and Researchers

This study may serve as a reference for researchers investigating programming education, gamification, automated programming assessment, learning analytics, machine learning, and AI-assisted educational-content generation. It may also provide locally collected data and implementation

experience that can inform future research, subject to the study's ethical and data-sharing restrictions. Building upon this foundation, future developers and researchers may extend ByteQuest by supporting additional programming languages, larger datasets, other predictive models, improved code assessment, more advanced learning-content generation, or more extensive evaluation of AI-generated educational materials.

Definition of Terms

The following terms are operationally defined according to their use in the study:

Automated Programming Assessment. The process of evaluating a student's submitted program using predefined test cases to determine whether the program satisfies specified requirements.

ByteQuest. The proposed gamified programming learning platform developed in this study for providing programming activities, AI-assisted learning-content generation, automated programming assessment, performance tracking, and machine-learning-based subsequent-task prediction.

Challenge. A programming activity that requires a student to apply programming concepts by producing a solution that can be evaluated through defined criteria or automated test cases.

F1-Score. A classification metric that combines precision and recall and will be used to evaluate the predictive models.

Gamification. The incorporation of game-related elements into the programming learning environment, such as points, badges, progress indicators, quests, challenges, and leaderboards.

Java. One of the programming languages included in the programming activities of ByteQuest.

Large Language Model (LLM). A pre-trained language-generation model used in ByteQuest to assist in generating quizzes, review questions, and programming challenges from retrieved content of uploaded programming materials. Gemini 2.5 Flash will serve as the selected LLM for this component.

Logistic Regression. A supervised machine-learning classification algorithm that will be used to predict the defined binary outcome of a subsequent programming task.

Machine Learning. A computational approach in which models learn patterns from data and use those patterns to make predictions or classifications on new observations.

NEMSU Computer Science Students. Selected students enrolled in the Bachelor of Science in Computer Science program of North Eastern Mindanao State University who will participate in the study.

Precision. The proportion of predicted positive outcomes that are actually positive according to the defined target outcome.

Programming Performance. The measurable results obtained by a student while completing programming-related activities, including quiz results and outcomes from programming tasks.

Programming Task. A structured programming activity requiring the student to develop a program or solution based on specified requirements.

Python. One of the programming languages included in the programming activities of ByteQuest.

Quiz. A structured set of questions used to assess a student's understanding of

programming concepts or material extracted from an uploaded learning resource.

Random Forest. A supervised machine-learning classification algorithm that combines multiple decision trees to produce a classification result.

Recall. The proportion of actual positive outcomes that are correctly identified by the predictive model.

Retrieval-Augmented Generation (RAG). An approach in which relevant information is retrieved from an external source and provided to a generative model as contextual information for producing an output. In ByteQuest, RAG is used to retrieve relevant portions of an uploaded programming document and provide them to Gemini 2.5 Flash for generating document-grounded learning content.

Subsequent Programming Task. The programming task that follows the activities used to obtain the predictor variables and whose defined outcome will serve as the target for the machine-learning prediction.

Test Case. A predefined input, expected output, or condition used by the automated assessment mechanism to determine whether a submitted program satisfies a specified requirement.

Training Dataset. The portion of the collected and prepared performance data

used to train a machine-learning model.

REFERENCES

Al Faraby, S., Adiwijaya, A., & Romadhony, A. (2024). Review on neural question generation for education purposes. *International Journal of Artificial Intelligence in Education*, *34*(3), 1008–1045. Retrieved from https://doi.org/10.1007/s40593-023-00374-x

Amnouychokanant, V., Boonlue, S., Chuathong, S., & Thamwipat, K. (2021). A study of first-year students' attitudes toward programming in the innovation in educational technology course. *Education Research International*, *2021*, Article 9105342. Retrieved from https://doi.org/10.1155/2021/9105342

Barczak, A. L. C., Mathrani, A., Han, B., & Reyes, N. H. (2023). Automated assessment system for programming courses: A case study for teaching data structures and algorithms. *Educational Technology Research and Development*, *71*, 2365–2388. Retrieved from https://doi.org/10.1007/s11423-023-10277-2

Borowitz, M. J., Blackford, A. L., Nagelia, S., & Hruban, R. H. (2026). Large language models can generate high-quality pathology multiple-choice questions comparable with questions written by a human expert. *Modern*

Pathology, *39*(1), Article 100940. Retrieved from https://doi.org/10.1016/j.modpat.2025.100940

Durak, A., & Bulut, V. (2024). Classification and prediction-based machine learning algorithms to predict students' low and high programming performance. *Computer Applications in Engineering Education*, *32*(1), Article e22679. Retrieved from https://doi.org/10.1002/cae.22679

Garcia, M. B., & Revano, T. F., Jr. (2021). Assessing the role of Python programming gamified course on students' knowledge, skills performance, attitude, and self-efficacy. In *2021 IEEE 13th International Conference on Humanoid, Nanotechnology, Information Technology, Communication and Control, Environment, and Management (HNICEM)* (pp. 1–5). Retrieved from https://doi.org/10.1109/HNICEM54116.2021.9731935

Gonzaga, A. S., & Badiang, R. O., Jr. (2025). Design and evaluation of a gamified learning tool. *International Journal of Education Research for Higher Learning*, *34*(1). Retrieved from https://doi.org/10.17158/kvkhzj12

Li, Z., Wang, Z., Wang, W., Hung, K., Xie, H., & Wang, F. L. (2025). Retrieval-augmented generation for educational application: A systematic survey. *Computers and Education: Artificial Intelligence*, *8*, Article 100417.

Retrieved from https://doi.org/10.1016/j.caeai.2025.100417

Zhan, Z., He, L., Tong, Y., Liang, X., Guo, S., & Lan, X. (2022). The effectiveness of gamification in programming education: Evidence from a meta-analysis. *Computers and Education: Artificial Intelligence*, *3*, Article 100096. Retrieved from https://doi.org/10.1016/j.caeai.2022.100096

Zhang, V., Jeffries, B., & Koprinska, I. (2025). A machine learning approach for predicting student progress in online programming education. *International Journal of Artificial Intelligence in Education*, *35*, 3614–3644. Retrieved from https://doi.org/10.1007/s40593-025-00510-9